Step 1 — A Single LCEL Chain That Answers One Question, No Tools — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 1: before any tool or loop, prove the LangChain pieces work on their own: settings load, the model answers, and an LCEL chain (`prompt | model | parser`) turns a question into plain text. **If not:** in Step 2 a failure could come from the chain, the tool or the loop, and you'd be debugging three things at once.

After this step the project runs: `python main.py` answers questions from the model's memory — no tools yet, so a live question like today's weather gets a guess.

The Doc01 files

These are Doc01's Build Task files (Intermediate, Approach 1), **unchanged** — only the file location and the `from src...` import line differ. They're the same three files as Project 1.

Story — `src/exceptions.py` : one file for every named error. Step 1 needs only Doc01's `MissingConfigError` ; Step 3 adds more. **If not:** errors would be scattered over many files, and a caller wouldn't know where to import them from.

```
# src/exceptions.py
# The project's named errors. MissingConfigError: a required setting is
# missing from .env. (Doc01's Build Task file.)

class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass
```

Story — `src/config.py` : the one place the project reads its settings from, failing loudly at startup. **If not:** a missing key would surface later as a confusing `AuthenticationError` .

```
# src/config.py
# Loads the settings from .env into a checked Config object. load_config()
# raises MissingConfigError when OPENAI_API_KEY is missing.
# (Doc01's Build Task file — only the import line changed.)
import os
```

```
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str

def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config() -> Config:
    load_dotenv()
    api_key = require_env("OPENAI_API_KEY")
    log_level = os.getenv("LOG_LEVEL", "INFO")
    return Config(openai_api_key=api_key, log_level=log_level)
```

Story — `src/utils/logger.py`: every file logs the same way, from one setup function. **If not:** each file would set up logging on its own, and some lines would print twice.

```
# src/utils/logger.py
# get_logger(name): one logging setup for every file — a timestamp, the
# file name and the level on every line. (Doc01's Build Task file.)
import logging
import os

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)

    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)

        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        formatter = logging.Formatter(log_format)
        handler.setFormatter(formatter)

        logger.addHandler(handler)
```

```
        logger.setLevel(level)

    return logger
```

Also create an empty `__init__.py` in `src/utils/`, so the folder is an importable package.

> Back to this step · Hint 1 · Hint 2 · Solution

Solution — a reusable chain, a model helper, and a prompt file

Story — `models/llm.py`: the one place that builds the `ChatOpenAI` model, with the key from `load_config()`. Every chain and agent in later steps gets its model from here. **If not:** each file would build its own model with its own settings, and changing the model would mean editing every one of them.

```
# src/models/llm.py
# The one place that builds the chat model: get_model(config) returns a
# ChatOpenAI with the project's model name and the key from Config.
from langchain_openai import ChatOpenAI

from src.config import Config

MODEL_NAME = "gpt-4o-mini"

def get_model(config: Config) -> ChatOpenAI:
    return ChatOpenAI(
        model=MODEL_NAME, temperature=0, api_key=config.openai_api_key
    )
```

Also create an empty `__init__.py` in `models/` and in `chains/`.

Story — `prompts/direct.txt` + `utils/prompts.py`: the prompt text lives in a `.txt` file, loaded by name — the same pattern as Project 1. The `{question}` placeholder is filled in by the chain. **If not:** the wording would be buried in Python code.

`src/prompts/direct.txt`

```
Answer this question from what you already know, in at most 2 sentences.

Question: {question}
```

```
# src/utils/prompts.py
# load_prompt(name): reads src/prompts/<name>.txt, so prompts live in
# text files instead of Python strings. (Project 1's file.)

PROMPTS_DIR = "src/prompts"
```

```
def load_prompt(name: str) -> str:
    path = PROMPTS_DIR + "/" + name + ".txt"
    with open(path) as prompt_file:
        return prompt_file.read().strip()
```

Story — `chains/direct_answer.py`: Doc05's `build_chain()` shape — prompt, model, parser — wrapped in a function so it can be imported. It stays in the project after this step: the compare script in Step 4 uses it to show how a from-memory answer differs from a tool-checked one. **If not:** the chain would be stuck inside `main.py`, and the whole point of the project (verified answers, not guesses) would have nothing to compare against.

```
# src/chains/direct_answer.py
# The no-tools chain: build_direct_chain(model) returns
# prompt | model | parser — it answers from the model's memory only.
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

from src.utils.prompts import load_prompt

def build_direct_chain(model: ChatOpenAI):
    prompt = ChatPromptTemplate.from_template(load_prompt("direct"))
    parser = StrOutputParser()
    return prompt | model | parser
```

Story — `main.py`: config first, then the model and the chain, then a question loop. A wrong key gets one clear line and a clean exit (Doc04's error rule). **If not:** a missing or wrong key would show a long traceback.

```
# main.py
# The file you run: python main.py
# Asks the no-tools chain each question and prints its answer from memory.
import sys

import openai

from src.chains.direct_answer import build_direct_chain
from src.config import load_config
from src.exceptions import MissingConfigError
from src.models.llm import get_model
from src.utils.logger import get_logger

def main() -> None:
    try:
        config = load_config()
```

```
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    model = get_model(config)
    chain = build_direct_chain(model)
    print("ResearchHand — ask a question, or /quit to exit.")

    while True:
        question = input("You: ").strip()
        if question == "/quit":
            break
        if question == "":
            continue

        try:
            answer = chain.invoke({"question": question})
        except openai.AuthenticationError:
            logger.error("Authentication failed — check OPENAI_API_KEY.")
            print("Login failed — check OPENAI_API_KEY in your .env file.")
            sys.exit(1)
        print(f"Answer (from memory): {answer}")

if __name__ == "__main__":
    main()
```

Run it: `python main.py`

Expected session (the model's wording varies):

```
ResearchHand — ask a question, or /quit to exit.
You: What is the capital of France?
Answer (from memory): The capital of France is Paris.
You: What's the weather in Lahore right now?
Answer (from memory): I don't have live data, but Lahore is usually
warm — around 30°C at this time of year.
You: /quit
```

(The long answer is wrapped here to fit the page.)

The second answer is exactly the problem this project solves: the model can only guess. Step 2 gives it a tool.

The Step 1 tests

Story — `tests/unit/test_config.py`: the same three checks as Doc01's `test_config.py` — a missing setting, an empty one, and a config that loads — with no API call. It tests `require_env()` with a made-up setting name, so your real `.env` is never moved or rewritten. **If not:** the first sign of a broken `.env` would be a confusing OpenAI error.

```
# tests/unit/test_config.py
# Checks the settings code with no API call. Each case prints OK or WRONG.
# It only changes settings inside this test run — your .env file is never
# touched.
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "RESEARCHHAND_TEST_SETTING"


def test_missing_setting() -> None:
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")


def test_empty_setting() -> None:
    os.environ[TEST_SETTING] = ""
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 2 (empty setting)   -> {result}")


def test_config_loads() -> None:
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 3 (config loads)    -> {result}")
```

```
if __name__ == "__main__":
    test_missing_setting()
    test_empty_setting()
    test_config_loads()
```

Story — `tests/integration/test_live_agents.py` : the checks that need the real model. It starts with one case — the no-tools chain answers a question — and every later step adds its own cases to this same file. **If not:** "it works" would only mean "it worked the one time I tried it by hand".

```
# tests/integration/test_live_agents.py
# Checks the app against the real OpenAI API (and Open-Meteo) — each step
# adds cases. Each case prints OK or WRONG.
# Needs a valid OPENAI_API_KEY in .env.
# Run from the repo root: python -m tests.integration.test_live_agents
from langchain_openai import ChatOpenAI

from src.chains.direct_answer import build_direct_chain
from src.config import load_config
from src.models.llm import get_model


def test_direct_answer(model: ChatOpenAI) -> None:
    chain = build_direct_chain(model)
    answer = chain.invoke({"question": "What is the capital of France?"})
    result = "OK"
    if "Paris" not in answer:
        result = "WRONG"
    print(f"case 1 (direct answer)  -> {result}: {answer}")


if __name__ == "__main__":
    config = load_config()
    model = get_model(config)
    test_direct_answer(model)
```

Also create the empty folders `tests/unit/` and `tests/integration/`. Tests run as modules, so they need no `__init__.py`.

Every test prints `OK` or `WRONG` — you never have to judge the output yourself. `WRONG` means: read that test function and the code it calls.

Run the tests (from the repo root):

```
python -m tests.unit.test_config
python -m tests.integration.test_live_agents
```

Expected output of `test_config`:

```
case 1 (missing setting) -> OK
case 2 (empty setting)   -> OK
case 3 (config loads)    -> OK
```

Expected output of `test_live_agents` (the model's wording varies):

```
case 1 (direct answer)  -> OK: The capital of France is Paris.
```